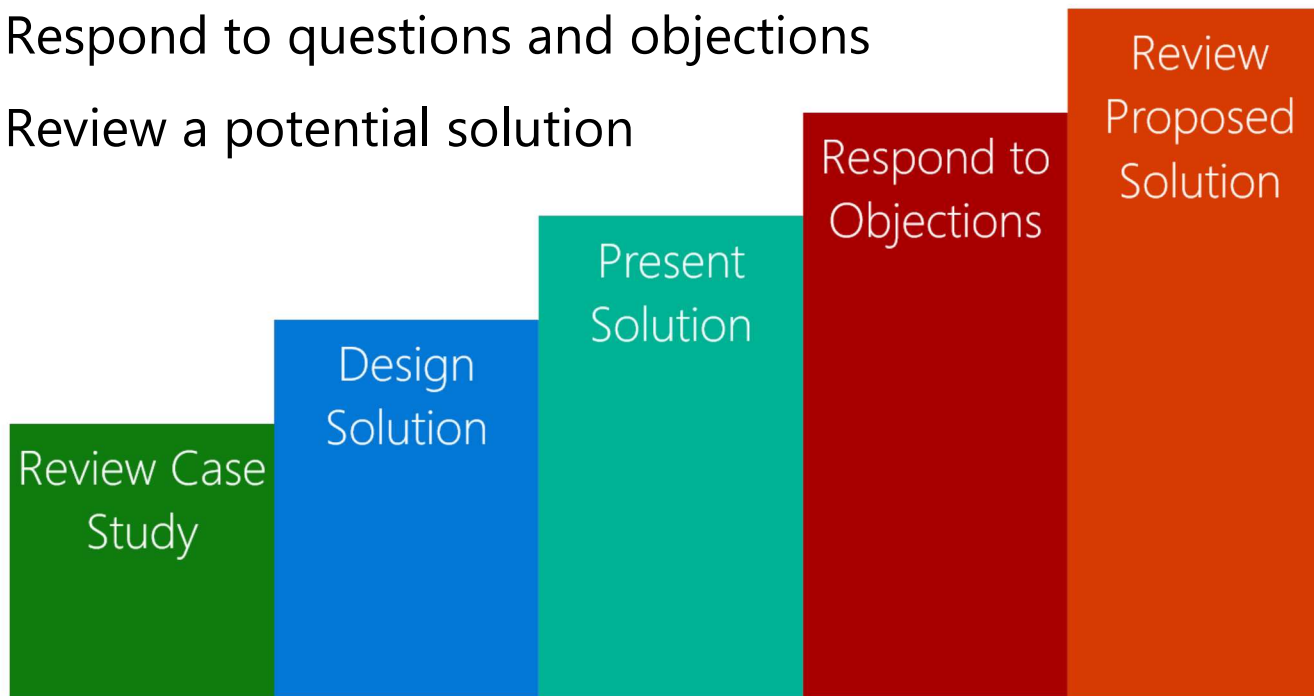


Building Microservice architecture - Case Study

- Case Study Overview
- Case Study Solution

Case Study Overview

- Review the case study requirements
- Design a solution to the customer business problem
- Present your solution
- Respond to questions and objections
- Review a potential solution



Customer Situation



Fabrikam Medical Conferences provides conference web site services tailored to the medical community.



After starting with a few small conferences, they now have evolved into a well-known brand and handle over 100 conferences per year, and growing.

Customer Situation



Each conference site has limited budget, but the conference owners have significant customization and change demands.



These changes can impact various aspects of the system from UI to back end, including conference registration and payment terms.

Customer Situation

- **12 developers handle**
 - **Development**
 - **Testing**
 - **Deployment**
 - **Operational management of all customer sites**
- **Due to customer demands, they have issues with the efficiency and reliability of their development and DevOps workflows.**

Customer Situation

- The technology used is the MEAN stack
 - Mongo, Express, Angular, and Node.js
 - Web sites and APIs are built as microservices hosted on Linux servers.
- The on-prem data backend is MongoDB; also running on a separate cluster of Linux servers.
- There is relational data stored in PostgreSQL running on Linux servers.

Customer Situation

- Conference owners (“customers”) are considered “tenants”, and each tenant is treated as a unique deployment including:
 - Each tenant has a database in the MongoDB cluster with its own collections, and a database in PostgreSQL.
 - A copy of the most recent functional conference code base is taken and configured to point at the tenant database
- Modifications are made to support the customer’s needs
 - The tenant’s code is deployed to a specific group of load balanced Linux server dedicated to one or more tenant.
 - Once the conference site is live, the inevitable requests for customizations to the deployment begins.



Customer Situation

- They are looking to achieve the following:
 - Reduce potential regressions introduced to functional tenant code when changes are made
 - Ideally, changes to individual areas should not require a full regression test of the site functionality
 - Reduce the time to onboard new tenants
 - Reduce overhead managing changes, and related deployments
 - Improve ability to roll back and recover post change
 - Increase visibility into system operations and health



Customer Needs

- Simplify new tenant deployment
- Improve reliability of tenant updates
- Choose a suitable Docker container strategy on Azure
- Migrate MongoDB data to Cosmos DB without application changes
- Migrate relational data from PostgreSQL on-premises databases to Microsoft Azure
- Continue to use Git repositories for source control
- Look at GitHub Actions as the CI/CD tool of choice

Customer Needs

- Use tools for deployment, CI/CD integration, container scheduling, orchestration, monitoring, and alerts
- They wish to complete an implementation of the proposed solution for a single tenant to train the team and perfect the process
- Enhance attendee session feedback with AI to prevent inappropriate content from being posted, and real-time language translation to better accommodate growing worldwide conference attendance.

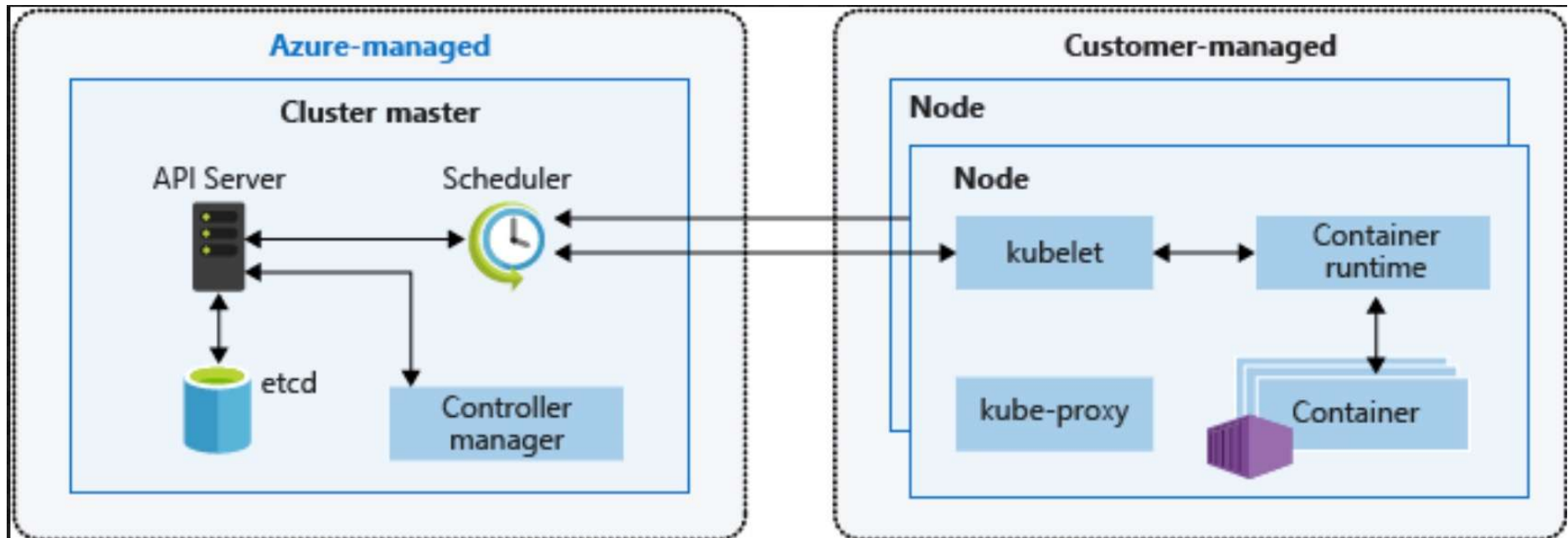
Customer Objection

- There are many ways to deploy Docker containers on Azure, how do those options compare and what are motivations for each?
- Is there an option in Azure that provides container orchestration platform features that are easy to manage and migrate to, that can also handle our scale and management workflow requirements?



Common Scenario

Kubernetes Architecture

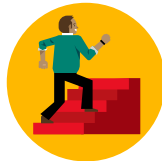


Call to Action

Case Study Timing: 60 Minutes



Who are the business decision makers and stakeholders?



What customer business needs do you need to address with your solution?



Diagram your proposed solution